

RAPPORT DE PROJET — PORTFOLIO

GLOBAL JERSEY STORE

Plateforme e-commerce full stack pour la vente de maillots de football des équipes nationales — développée comme projet phare de portfolio développeur web.

Livré à Nossaiiba El Asri · Juillet 2026 · Next.js 15 · React 19 · TypeScript · Firebase

1. PRÉSENTATION DU PROJET

Global Jersey Store est une application e-commerce complète permettant d'acheter des maillots officiels de style équipes nationales de football (16 pays). Le site couvre l'intégralité d'un parcours d'achat réel : recherche instantanée, filtres avancés, fiche produit détaillée, panier, wishlist, authentification, et un dashboard admin pour gérer le catalogue.

Le projet a été pensé pour démontrer, dans un contexte de recrutement, une maîtrise réelle de l'écosystème React/Next.js moderne : architecture propre, composants réutilisables, gestion d'état, formulaires validés, responsive design et déploiement.

OBJECTIFS

- Construire une application **full stack** de qualité production, pas un simple prototype.
- Démontrer une architecture **scalable** et **maintenable** (séparation UI / logique / services).
- Couvrir les compétences les plus recherchées par les recruteurs : TypeScript, state management, auth, base de données, responsive, accessibilité.
- Créer une identité visuelle forte et différenciante (thème sombre, façon boutique de sport premium).



2. STACK TECHNIQUE

CATÉGORIE	TECHNOLOGIE	RÔLE
Framework	Next.js 16 (App Router)	Rendu hybride (Server/Client Components), routing, structure des pages
UI Library	React 19	Composants, hooks, interactivité
Langage	TypeScript	Typage statique, sécurité du code, autocomplétion
Styling	Tailwind CSS v4	Design system utilitaire, thème clair/sombre
Composants UI	shadcn/ui	Composants accessibles (dialog, select, tabs, sheet...)
Icônes	Lucide Icons	Iconographie cohérente
Animations	Framer Motion	Transitions, hero animé, micro-interactions
State management	Zustand	Panier et wishlist persistés (localStorage)
Formulaires	React Hook Form + Zod	Formulaires validés (login, contact, admin)
Backend / Auth / DB	Firebase (Auth, Firestore, Storage)	Authentification, base de données, stockage fichiers
Déploiement	Vercel	Hébergement recommandé pour Next.js

POURQUOI CETTE STACK ?

C'est la combinaison la plus demandée aujourd'hui pour un poste de développeur front-end/full-stack React. Next.js + TypeScript + Tailwind + shadcn/ui est devenu un standard de facto en 2025-2026, et Firebase permet de montrer une vraie intégration backend sans avoir à gérer un serveur soi-même.

3. ARCHITECTURE DU PROJET

Architecture en couches, avec séparation claire entre **UI** (components/ui), **logique métier par domaine** (features/), **données** (constants/, types/) et **services** (lib/).

```
global-jersey-store/
├─ app/                → Routes (App Router) : home, shop, product, cart, login, profile, admin...
├─ components/
│  ├─ ui/              → Composants shadcn/ui bruts (button, card, dialog...)
│  ├─ layout/          → Header, Footer
│  └─ shared/          → Composants réutilisés (ProductCard, Logo, SearchBar...)
├─ features/           → Logique + UI groupées par domaine (home/, shop/, cart/, product/, admin/, auth/)
├─ constants/          → Données produits et équipes (teams.ts, products.ts)
├─ lib/
│  ├─ firebase.ts      → Config Firebase (Auth/Firestore/Storage)
│  ├─ store/           → Zustand stores (cart-store.ts, wishlist-store.ts)
│  └─ utils.ts, format.ts
├─ types/              → Interfaces TypeScript (Product, Team)
└─ public/             → Assets statiques
```

MODÈLE DE DONNÉES — PRODUCT

```
interface Product {
  id: string
  name: string
  slug: string
  country: string
  brand: string
  price: number
  compareAtPrice?: number
  sizes: string[]
  description: string
  images: string[]
  stock: number
  rating: number
  reviews: Review[]
  isFeatured: boolean
  category: "home" | "away"
  isTrending: boolean
  stillInCompetition: boolean
}
```

Base de données : le projet utilise Firebase (Firestore) comme backend de production. Pour que la démo fonctionne immédiatement sans qu'on ait besoin de créer un projet Firebase, le catalogue (16 équipes × 2 maillots) est actuellement défini comme dataset local dans `constants/teams.ts` et `constants/products.ts` — exactement la même forme de données que celle qui serait stockée dans Firestore. La configuration Firebase

(`lib/firebase.ts`) est déjà prête : il suffit d'ajouter tes vraies clés dans `.env.local` et de migrer ce dataset vers une collection Firestore pour passer en production.

4. PAGES & FONCTIONNALITÉS

PAGE	CONTENU
Home	Hero animé, Trending Jerseys (équipes encore en compétition), Best Sellers, Categories, Newsletter
Shop	Catalogue complet, filtres (pays, taille, prix, marque), tri, pagination
Product	Galerie images, tailles, stock, ajout panier/wishlist, produits similaires
Cart	Liste, quantités, sous-total, taxes, total, checkout simulé
Login	Connexion / création de compte / Google Login (Firebase Auth)
Profile	Infos, historique commandes, favoris
Wishlist	Favoris enregistrés
About / Contact	Présentation de la marque, formulaire de contact validé
Admin Dashboard	CRUD produits, gestion stock, commandes

FONCTIONNALITÉS TRANSVERSES

- ✓ Recherche instantanée
- ✓ Filtres multi-critères
- ✓ Wishlist persistée (Zustand + localStorage)
- ✓ Panier persistant avec calcul auto des totaux
- ✓ Thème sombre par défaut (+ bascule clair/sombre)
- ✓ Responsive mobile-first (testé 390px → 1440px)
- ✓ Animations Framer Motion (hero, cartes, transitions)
- ✓ Toast notifications (Sonner)
- ✓ Skeleton loading
- ✓ Pagination catalogue
- ✓ Section "Still Competing" dynamique selon l'actualité (données simulées)

5. DIRECTION ARTISTIQUE

Le design s'inspire des grandes boutiques de sport (Nike, Adidas, WorldSoccerShop) : thème sombre premium, typographie bold en majuscules, un unique accent gold pour hiérarchiser l'information (badges, prix en vedette, ratings), rouge comme couleur de marque principale.

#0a0b0d — Fond

#e5122f — Rouge primaire

#d4a017 — Gold accent

VISUELS PRODUITS — NOTE IMPORTANTE

Sur les visuels de maillots : j'ai cherché des vraies photos officielles issues de sites comme WorldSoccerShop, Nike, Fanatics — mais ces plateformes bloquent activement la récupération automatisée d'images (protection anti-bot) et ces photos sont surtout protégées par le droit d'auteur du fabricant (Nike/Adidas/Puma) et de la fédération. Les utiliser sans licence dans un vrai produit public serait risqué.

Le catalogue utilise donc actuellement des visuels générés reprenant fidèlement les vrais codes de chaque maillot (rayures Argentine, checkerboard Croatie, cols contrastés, écusson) — un compromis sûr légalement, mais qui reste stylisé plutôt que photographique.

Recommandation : pour un rendu 100% photo-réaliste, deux options : (1) acheter un pack de photos stock de maillots génériques (sites comme Envato, Freepik) et les recolorer, ou (2) si ce projet reste un portfolio privé (non commercial), tu peux légalement télécharger toi-même quelques photos de maillots depuis ces sites et me les envoyer — je les intégrerai directement au projet.

6. LANCER LE PROJET

```
cd global-jersey-store
npm install
npm run dev
→ ouvre http://localhost:3000
```

VARIABLES D'ENVIRONNEMENT (OPTIONNEL — POUR ACTIVER FIREBASE RÉEL)

Crée un fichier `.env.local` à la racine :

```
NEXT_PUBLIC_FIREBASE_API_KEY=...
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=...
NEXT_PUBLIC_FIREBASE_PROJECT_ID=...
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=...
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=...
NEXT_PUBLIC_FIREBASE_APP_ID=...
```

Sans ces clés, le site fonctionne quand même normalement (catalogue, panier, wishlist) — seules l'authentification réelle et Firestore nécessitent ces identifiants.

DÉPLOIEMENT (VERCEL)

- Pousser le projet sur un repo GitHub
- Importer le repo sur vercel.com
- Ajouter les variables d'environnement Firebase dans les Project Settings
- Deploy — Vercel détecte automatiquement Next.js

7. CONCLUSION & PERSPECTIVES

Le projet **Global Jersey Store** atteint son objectif initial : une application e-commerce complète et cohérente, couvrant l'ensemble du parcours utilisateur (catalogue, fiche produit, panier, authentification, wishlist, back-office) sur une architecture propre et typée de bout en bout.

ÉTAT ACTUEL

COMPOSANT	STATUT
Frontend (UI/UX, pages, navigation)	Terminé
State management (panier, wishlist)	Terminé — persistant côté client
Formulaires & validation	Terminé
Intégration Firebase (Auth / Firestore réels)	Configurée, à activer avec de vraies clés
Tests automatisés	Non couvert — à ajouter

CI/CD	En place (GitHub Actions — lint + build à chaque push)
Déploiement production	À réaliser (Vercel recommandé)

LIMITES CONNUES

- Le catalogue est actuellement un dataset statique (constants/) plutôt qu'une vraie collection Firestore — migration nécessaire avant mise en production réelle.
- Les visuels produits sont stylisés plutôt que des photographies sous licence (voir section 5).
- Aucune suite de tests automatisés n'est en place à ce stade.

PROCHAINES ÉTAPES RECOMMANDÉES

1. Connecter un vrai projet Firebase (Auth + Firestore) et migrer le dataset produits.
2. Ajouter une suite de tests (Vitest pour l'unitaire, Playwright pour l'E2E).
3. Déployer en production sur Vercel, connecté au dépôt GitHub.
4. Sécuriser les règles Firestore (accès admin vs client) avant ouverture publique.